

Intelligenza Artificiale

“Deep learning (part 2)”

Nicola Bellotto

A.A. 2025/2026

Lecture outline

- Inputs and outputs
- Learning NNs
- Back-propagation
- Recurrent Neural Networks



Slides partly based on Russell & Norvig instructors material <http://aima.cs.berkeley.edu/instructors.html>

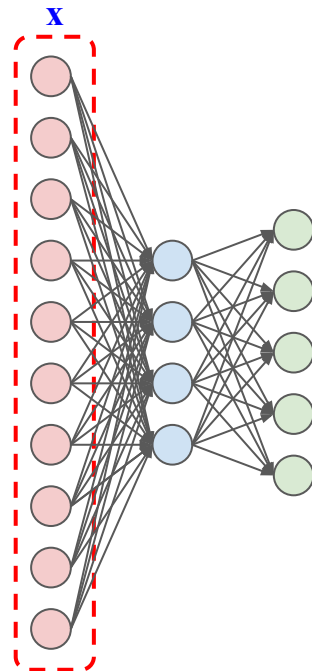
Input encoding

- Input units connect directly to the input data **x**, which could be
 - **Boolean** – **false** is mapped to input **0** and **true** is mapped to **1** (or **-1**, **+1**)
 - **Numeric** – attributes are used directly, with no need for mapping (except maybe some scaling/normalisation)
 - **Categorical** – attributes with more than two values (i.e. not Boolean) are usually represented with **one-hot encoding**
 - an attribute with d possible values is represented by d separate input bits
 - corresponding input bit is set to **1** and others **0**

• e.g.

<i>French</i>	<i>Italian</i>	<i>Thai</i>	<i>Burger</i>
0	1	0	0

Not represented by simple integers, otherwise the type *French* could be $>$ *Italian*, which doesn't make sense...



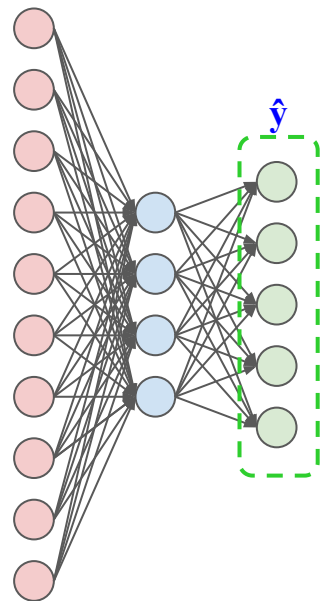
Output layers for regression

- For **regression** problems, use **linear output layer**
 - that is, take the layer's input as it is, without activation functions:

$$\hat{y}_j = \sum_i w_{i,j} a_i$$

interpreted as the mean of a Gaussian prediction with fixed variance

- For many other deep learning applications though, it's common to interpret the output values $\hat{y}$ as probabilities
 - easy for a single Boolean output: just use a sigmoid



Output layers for classification

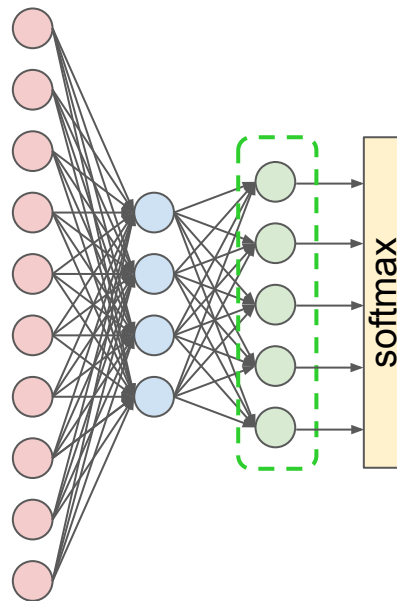
- For (multiclass) **classification**, output probabilities must sum to **1**

⇒ use a **softmax layer**

- outputs a vector of d values (probabilities)
given a vector of inputs $in_1, \dots, in_d$

$$\text{softmax}(\mathbf{in})_k = \frac{e^{in_k}}{\sum_{k'=1}^d e^{in_{k'}}}$$

- the output is a vector of non-negative numbers that **sum to 1**
 - e.g. if input vector of **softmax** is $\langle 5, 2, 0, -2 \rangle$
→ **softmax** output $\langle 0.946, 0.047, 0.006, 0.001 \rangle$



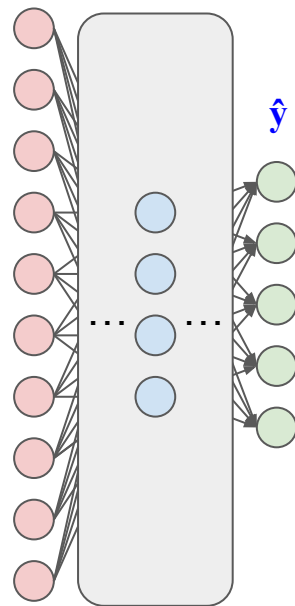
Output layers and loss functions

- When output values $\hat{\mathbf{y}}$ interpreted as probabilities (for classification)
 - the difference between the *true* distribution $\mathbf{y}$ and the *predicted* distribution $\hat{\mathbf{y}}$ is measured by a **cross-entropy loss**
 - usually approximated by a **negative log likelihood** loss (**log loss**)
 - training the network (weights) corresponds to minimising such loss

$$\mathbf{w}^* = \underset{\mathbf{w}}{\operatorname{argmin}} - \sum_{j=1}^N \log P_{\mathbf{w}}(\mathbf{y}_j | \mathbf{x}_j)$$

- this is equivalent to maximizing the probability of the observed data

$$\operatorname{argmax}_{\mathbf{w}} \sum_j \log P_{\mathbf{w}}(\mathbf{y}_j | \mathbf{x}_j) = \operatorname{argmax}_{\mathbf{w}} \prod_j P_{\mathbf{w}}(\mathbf{y}_j | \mathbf{x}_j)$$

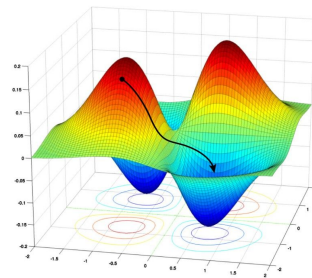


Learning algorithms

- Neural networks are almost always trained with some variant of (minibatch) **stochastic gradient descent**, or **SGD**

- The goal is to minimize the loss $L(\mathbf{w})$, where $\mathbf{w}$ represents all of the parameters of the network

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla L(\mathbf{w})$$

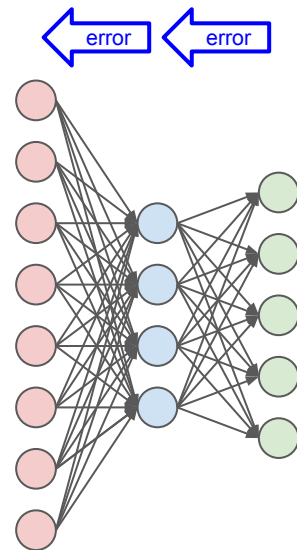


Back-propagation

- How to calculate the gradient?

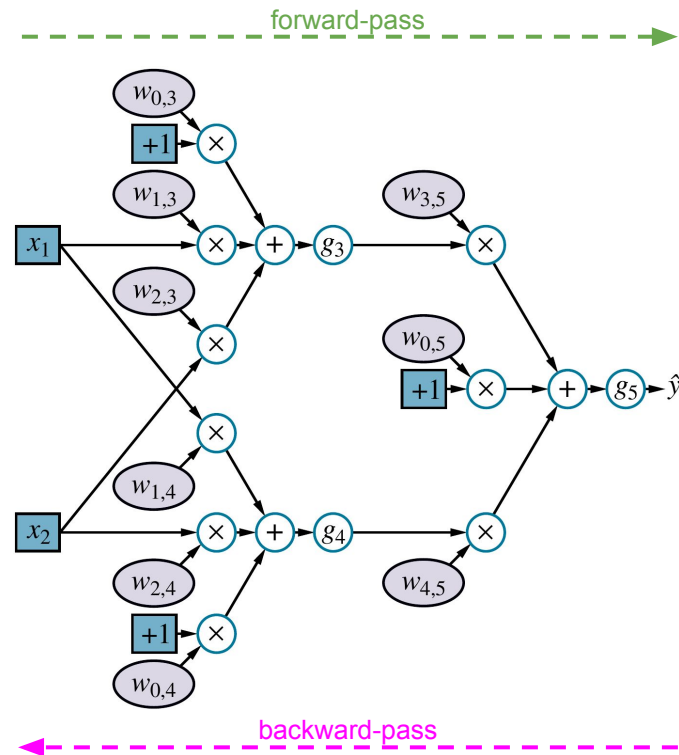
Idea based on

- **Back-propagation**: output errors are passed back through the network
- Starting from **m = output layer**
 1. Compute the errors of the units in layer **m**
 2. Find how much each unit in layer **$m - 1$** contributed to those errors
 3. Update the relative weight between units
 4. Repeat for **$m \leftarrow m - 1$** until **input layer**



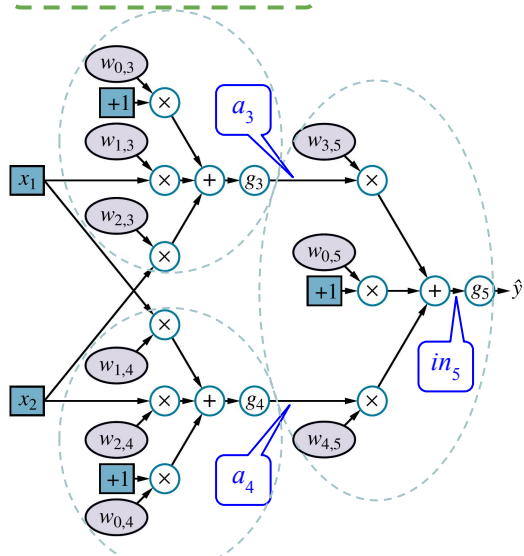
SGD with back-propagation

- To apply SGD, we need to compute the gradient $\partial L / \partial w$ for each weight w
(consider the computation graph)
- This is done by **back-propagation** in two passes:
 - During a **forward-pass**, each node computes some arbitrary function (*multiplication, addition, activation*) from its inputs, which come from previous nodes
 - ← The gradient is computed by a **backward-pass**, propagating error information (i.e. *loss*) from the output layer of the network to the hidden layers



Example: computing $\partial L/\partial w$ with squared loss

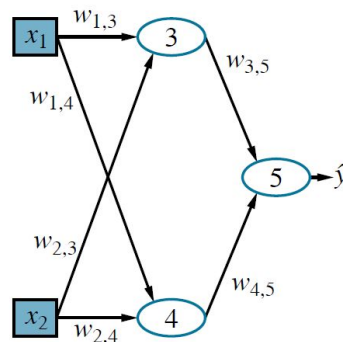
$$\begin{aligned}
 \frac{\partial}{\partial w_{3,5}} \text{Loss}(h_w) &= \frac{\partial}{\partial w_{3,5}} (y - \hat{y})^2 = -2(y - \hat{y}) \frac{\partial \hat{y}}{\partial w_{3,5}} \\
 &= -2(y - \hat{y}) \frac{\partial}{\partial w_{3,5}} g_5(\text{in}_5) = -2(y - \hat{y}) g'_5(\text{in}_5) \frac{\partial}{\partial w_{3,5}} \text{in}_5 \\
 &= -2(y - \hat{y}) g'_5(\text{in}_5) \frac{\partial}{\partial w_{3,5}} (w_{0,5} + w_{3,5}a_3 + w_{4,5}a_4) \\
 &= -2(y - \hat{y}) g'_5(\text{in}_5) a_3.
 \end{aligned}$$



- The weight $w_{3,5}$ can be updated by

$$w_{3,5} \leftarrow w_{3,5} - \alpha \partial \text{Loss} / \partial w_{3,5}$$

- The remaining weights can be updated using the other partial derivatives



Back-propagation messages

- **Back-propagation** can be thought has a process where **messages** are passed back along each link in the graph

- Messages are partial derivatives of the loss L

- **Incoming messages** of internal nodes are summed...

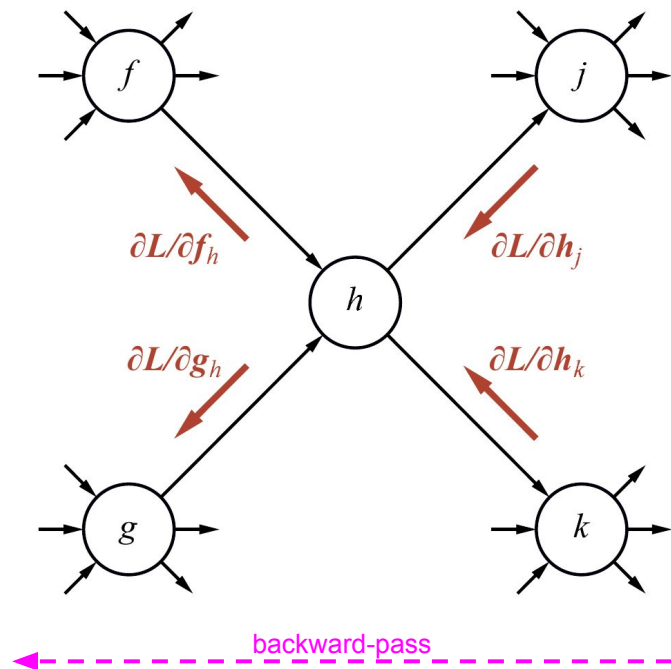
$$\partial L / \partial h = \partial L / \partial h_j + \partial L / \partial h_k$$

- ... then multiplied to local partial derivatives to compute the **outgoing messages**

$$\partial L / \partial f_h = (\partial L / \partial h) (\partial h / \partial f_h) \quad \partial L / \partial g_h = (\partial L / \partial h) (\partial h / \partial g_h)$$

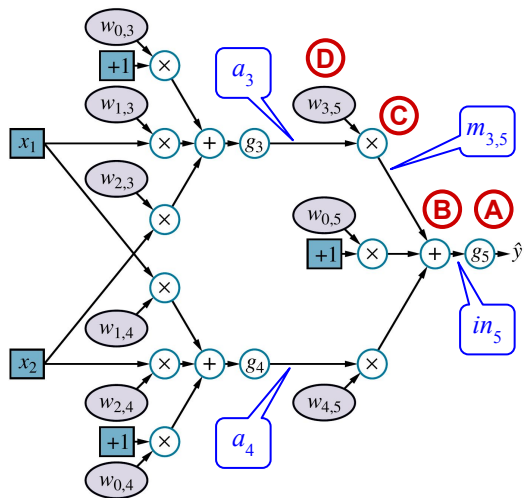
- For example, if h is a multiplication node, then $h = f \times g$ and $\partial h / \partial f = g$, $\partial h / \partial g = f$

- The process starts at the output nodes with $\partial L / \partial \hat{y}$ and terminates at the weight nodes with $\partial L / \partial w$



Example: computing $\partial L / \partial w$ with squared loss

$$\begin{aligned}
 \frac{\partial}{\partial w_{3,5}} \text{Loss}(h_w) &= \frac{\partial}{\partial w_{3,5}} (y - \hat{y})^2 = -2(y - \hat{y}) \frac{\partial \hat{y}}{\partial w_{3,5}} \\
 &= -2(y - \hat{y}) \frac{\partial}{\partial w_{3,5}} g_5(in_5) = -2(y - \hat{y}) g'_5(in_5) \frac{\partial}{\partial w_{3,5}} in_5 \\
 &= -2(y - \hat{y}) g'_5(in_5) \frac{\partial}{\partial w_{3,5}} (w_{0,5} + w_{3,5} a_3 + w_{4,5} a_4) \\
 &= -2(y - \hat{y}) g'_5(in_5) a_3.
 \end{aligned}$$



On the left is the analytical form, which can be computed **algorithmically** by using messages:

- A)** $\partial L / \partial g_5 = \partial L / \partial \hat{y} = (y - \hat{y})^2 / \partial \hat{y} = -2(y - \hat{y})$
 - B)** $\partial L / \partial in_5 = (\partial L / \partial g_5) (\partial g_5 / \partial in_5) = (\partial L / \partial g_5) g'_5(in_5)$
 - C)** Let's call $m_{3,5} = w_{3,5} a_3$
then $in_5 = m_{3,5} + w_{0,5} + w_{4,5} a_4$
 - D)** Since $\partial m_{3,5} / \partial w_{3,5} = a_3$
- $$\begin{aligned}
 \partial L / \partial w_{3,5} &= (\partial L / \partial m_{3,5}) (\partial m_{3,5} / \partial w_{3,5}) = (\partial L / \partial m_{3,5}) a_3 \\
 &= -2(y - \hat{y}) g'_5(in_5) a_3
 \end{aligned}$$

NOTE: all the a_i and in_j quantities are computed during the forward-pass

Pros & cons of back-propagation

Pros

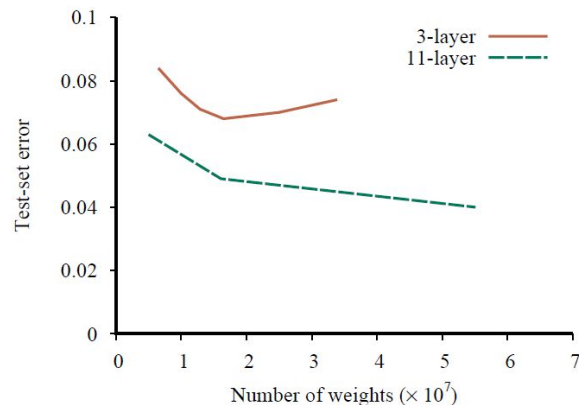
- 👍 Computation cost is linear in the number of nodes of the computation graph
- 👍 All of the gradient of activation functions can be prepared in advance and compiled into very efficient code for each node in the graph

Cons

- 😞 Requires storing most of the intermediate values (e.g. a output of each unit) that were computed during forward-pass in order to calculate gradients in the backward-pass
- 😞 Can suffer from **vanishing gradient problem**: if gradients $g'(\cdot) \ll 1$ (very likely with **sigmoid** or **tanh**) then back-propagated errors have little effect on the weights update

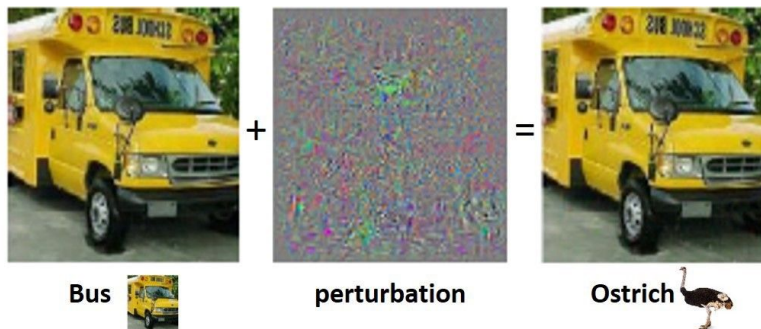
Generalization of deep networks

- Deep learning systems perform better than any other pure machine learning approaches for high dimensional inputs (images, video, speech signals, etc)
- When comparing networks with similar numbers of weights, deeper networks usually generalize better
 - E.g. figure shows test-set error with CNN for house number recognition



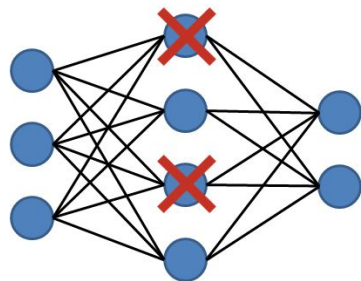
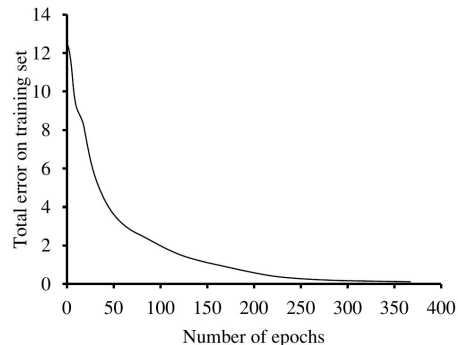
Adversarial attacks

- However, CNNs and other networks can fail in case of **adversarial examples**
 - random noise added to an image can produce completely unexpected results
 - e.g.



Dropout

- ➔ In general, good training results do not mean good performance in the real-world
 - The output error decreases with number of training epochs, but this might not be the case on the **testing** set (e.g. with adversarial examples)
- **Dropout** is a possible solution that “disable” some of the units during the training process (e.g. after each minibatch)
- This reduces the test error by making it harder to fit the training set
 - the model is forced to become robust to noise
 - approximates a large ensemble of thinned networks
 - applied to later layers, forces the final decision to pay attention to all of the abstract features of the example

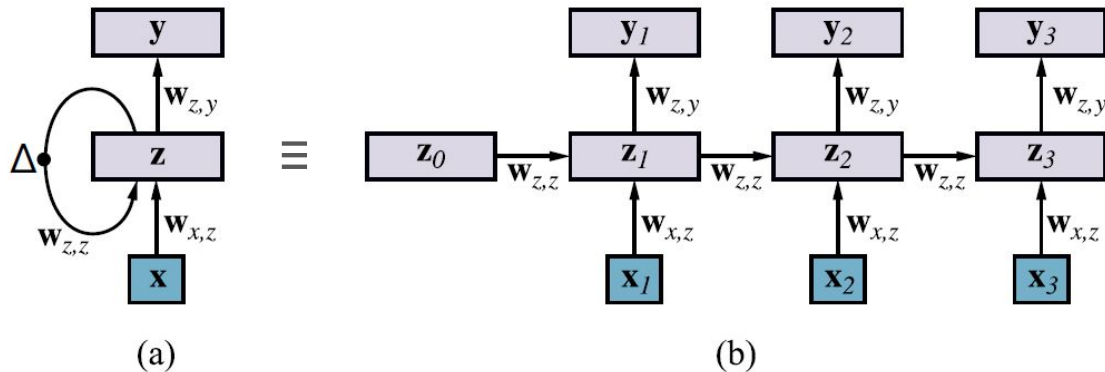


Recurrent Neural Networks

- **Recurrent neural networks (RNNs)** allow cycles in the computation graph
 - each cycle has a delay
 - units may take as input a value computed from their own output at an earlier step
- RNN maintain an internal **state/memory**
 - greater expressive power compared to feedforward networks

(a) RNN with delay Δ

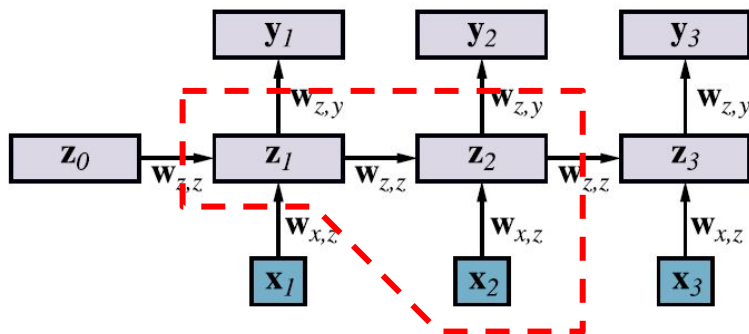
(b) “Unrolled” version over 3 time steps



Markov assumption of RNNs

- For the hidden state $\mathbf{z}_t$ of an RNN we make a **Markov assumption**
 - $\mathbf{z}_t$ captures all the information from previous inputs at time $t - 1$

$$\mathbf{z}_t = f_{\mathbf{w}}(\mathbf{z}_{t-1}, \mathbf{x}_t)$$

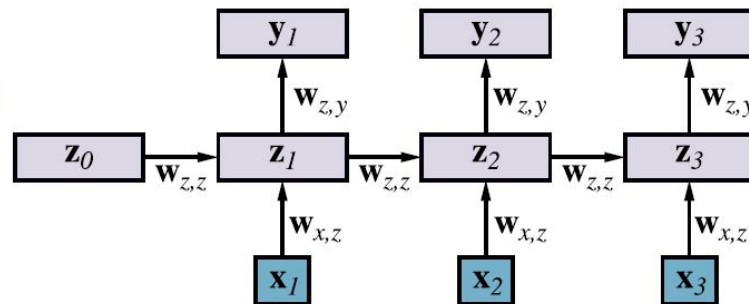


Training a basic RNN

- Case with input layer $\mathbf{x}$, hidden layer $\mathbf{z}$ with recurrent connections, and output layer $\mathbf{y}$
- $\mathbf{g}_z$ and $\mathbf{g}_y$ denote the activation functions for the hidden and output layers, respectively

$$\mathbf{z}_t = f_{\mathbf{w}}(\mathbf{z}_{t-1}, \mathbf{x}_t) = \mathbf{g}_z(\mathbf{W}_{z,z}\mathbf{z}_{t-1} + \mathbf{W}_{x,z}\mathbf{x}_t) \equiv \mathbf{g}_z(\mathbf{in}_{z,t})$$

$$\hat{\mathbf{y}}_t = \mathbf{g}_y(\mathbf{W}_{z,y}\mathbf{z}_t) \equiv \mathbf{g}_y(\mathbf{in}_{y,t}),$$



- The gradient of the loss (summed over all the timesteps) can be computed as usual, only paying attention to the gradient of the hidden unit $\mathbf{z}_t$, which can be shown to be recursive

$$\partial z_t / \partial w_{z,z} = g'_z(in_{z,t}) (z_{t-1} + w_{z,z} \partial z_{t-1} / \partial w_{z,z})$$

Training a basic RNN

- **Backpropagation through time (BPTT):**
 - The contribution to the gradient at time step t is calculated using the contribution from time step $t - 1$
 - The total runtime for computing the gradient will be linear in the size of the network
- Possible problems
 - If $w_{z,z} < 1$, a simple RNN can suffer from the **vanishing gradient problem**
 - If $w_{z,z} > 1$, we may experience the **exploding gradient problem**

Conclusions

- Suggested reading
 - Chapter 22, sec. 22.3–22.6 of Russell & Norvig
- Next lecture
Deep learning (part 3)
- Questions?

